CityClean-Agent 启动说明

投递包解压后，所有启动命令都在 04-source/CityClean-Agent/ 目录执行。不要直接在 ZIP 内运行。观看视频和阅读 PDF 不需要安装依赖。

方式一：Docker Compose（推荐）

要求 Docker Engine 正常运行，并且可以联网下载镜像与依赖。Windows 可使用 Docker Desktop 的 Linux containers 模式；无需另外安装 Python、Node.js 或 Conda，也无需 GPU。

在源码目录打开终端：

```
docker version
docker compose up -d --build
docker compose ps
```

首次构建需要联网，耗时取决于下载速度。docker version 应同时显示 Client 和 Server；只显示 Client 或报 named pipe 错误时，先启动 Docker Desktop 并等待 Engine 就绪。

服务 healthy 后打开：

- 工作台：http://127.0.0.1:8088/

- 健康检查：http://127.0.0.1:8088/api/v1/health

默认模式提供规则研判、合成路网、车辆调度、独立校验和执行重规划，不需要 API Key。Compose 只对本机开放 8088 端口。

查看日志与停止：

```
docker compose logs --tail=100 api web
docker compose down
```

仿真会话位于内存，服务重启后需要重新创建。当前 Compose 未使用数据库卷。

方式二：本地开发

需要 Conda、Python 3.11+ 和 Node.js。当前验证环境为 Python 3.12.13、Node.js 24.15.0；Vite 8 要求 Node.js 20.19+ 或 22.12+。项目约定 Conda 环境名称为 langchain1.2。

如果该环境尚不存在，先创建一次：

```
conda create -n langchain1.2 python=3.12 -y
```

终端一，在源码根目录安装并运行后端：

```
conda activate langchain1.2
python -m pip install -e ".[dev]"
python -m uvicorn cityclean_agent.api.app:app --host 127.0.0.1 --port 8000
```

终端二，从源码根目录运行前端：

```
cd frontend
npm ci
npm run dev -- --host 127.0.0.1
```

打开 http://127.0.0.1:5173/。前端开发服务器将 /api 代理至本机 8000 端口。API 文档位于 http://127.0.0.1:8000/docs。分别在两个终端按 Ctrl+C 停止服务。

.env 与 frontend/.env.local 均为可选配置，默认演示无需创建。npm ci 使用随源码提供的 package-lock.json；后端版本范围以 pyproject.toml 为准，已验证版本见验证记录。

三分钟演示流程

1. 进入工作台，展开“实验参数与完整结果”，将场景种子设置为 20260925，点击“重新生成场景”。资源使用 30 个候选、4 辆车、每车 1600 升、最多治理 10 个点。

2. 点击操作台中的“运行规则研判”，等待出现治理策略和优先目标。

3. 点击“生成当前策略方案”，等待出现“独立校验通过”；固定演示场景分配 10 个任务。

4. 点击“启动执行仿真”，再点击一次“推进 5 分钟”。

5. 点击“道路封闭”，观察封闭路段、重规划结果和变化后的路线。

6. 使用“自动运行”或继续步进，查看车辆、任务完成数和事件时间线。扰动后若显示需要人工处理，应结合容量、可达性和时限判断，不能将其视为成功。

视频展示的是这条规则模式链路。实时 DeepSeek A/B 需要另外配置凭据，不属于默认演示或自动测试。

可选：真实 DeepSeek 与天地图

真实 DeepSeek 调用需要 API Key 并会产生服务商费用。当前模型名为 deepseek-v4-flash，实际可用性由服务商和账号决定。

本地开发：将根目录 .env.example 复制为 .env，仅在本机填写 DEEPSEEK_API_KEY，再启动后端。页面中主动点击“使用 DeepSeek 研判”才会请求模型。

Compose：通过宿主机环境变量或根目录未提交的 .env 提供 DEEPSEEK_API_KEY，然后执行 docker compose up -d 更新服务环境。不要将凭据写入 Dockerfile、源码或投递附件。

天地图为可选显示层。开发模式通过 frontend/.env.local 设置 VITE_TIANDITU_KEY；Compose 通过构建前的宿主机环境变量或根目录 .env 设置该变量，然后重建 web。Vite 会将它写入浏览器产物，因此它是客户端 Key，应在服务商侧限制来源。无 Key 时显示合成路网、风险点、路线和车辆。

验证命令

在源码根目录执行：

```
conda activate langchain1.2
$env:PYTHONUTF8 = "1"
python -m pytest -q
python -m cityclean_agent.benchmark
```

前端检查：

```
cd frontend
npm run build
```

默认测试及 Benchmark 不访问真实模型。Benchmark 输出写入源码根目录的 artifacts/evaluations/，不作为产品运行输入。Windows 验证前设置 PYTHONUTF8=1，让测试的 Python 子进程也使用 UTF-8；仅给父进程加 -X utf8 不会传给子进程。Windows 如遇 Conda 包装层输出编码错误，可使用 conda run --no-capture-output -n langchain1.2 python -m cityclean_agent.benchmark。

常见问题

现象	检查方式
工作台提示无法连接服务	确认 api 已启动；检查健康接口、8088/8000 端口和 docker compose ps
默认 Runtime 或设施状态 unavailable/503	默认没有装配权威 Runtime 与设施仓储；使用合成实验操作台即可
页面无法显示真实底图	天地图 Key 可选；默认合成路网仍可演示
DeepSeek 报 Key 未配置或调用失败	检查本机凭据、账号模型权限和网络；规则模式无需这些配置
容器构建依赖下载超时	核实网络；可按仓库 README 的单次 build-arg 包源示例构建 api
8088、8000 或 5173 被占用	先确定占用进程；不要批量停止无关服务。若修改 API 端口，同步修改 Vite 代理配置
重启后仿真会话不存在	当前会话仅保存在内存；重新生成场景并启动仿真

完整技术说明与边界见《项目说明》及源码 README。